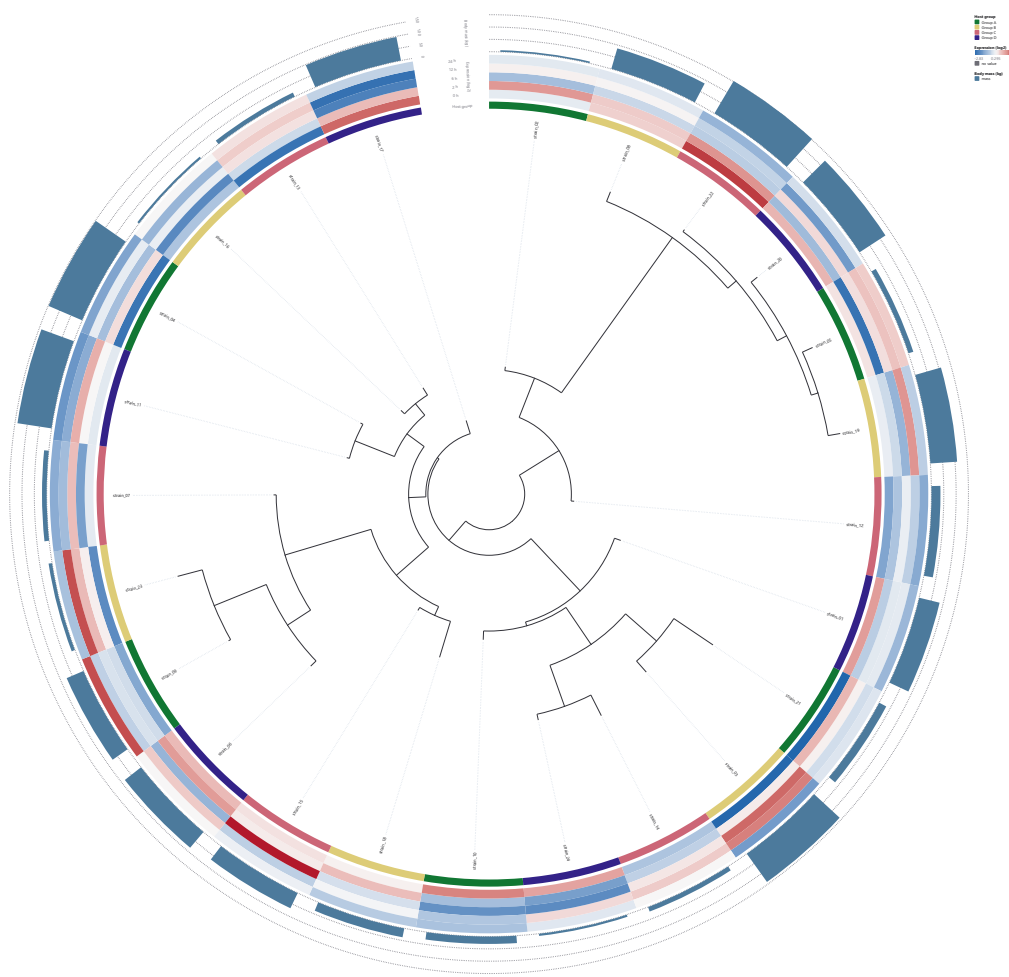


MakeYourTree

Phylogenetic tree visualisation and annotation

User Manual

Version 0.1.0



Manish Prakash Victor

Free and open-source software under the MIT licence
No server. No account. No network access at runtime.

MakeYourTree User Manual

Version 0.1.0, for MakeYourTree 0.1.0.

Copyright © 2026 Manish Prakash Victor.

MakeYourTree is free and open-source software, released under the MIT licence. You may use, copy, modify and redistribute it, including commercially. The software is provided without warranty of any kind. The full text is in `LICENSES/MIT.txt` in the source repository.

This manual is released under the same terms. Every figure in it was produced by MakeYourTree itself, from the synthetic datasets in `examples/`, by running `python tools/make_gallery.py`.

MakeYourTree is an independent implementation. It is not affiliated with, endorsed by, or derived from any other phylogenetics package.

The desktop application links PySide6/Qt 6, which is distributed under the LGPL-3.0. If you redistribute a compiled bundle, that licence places conditions on you; see `LICENSE.md` in the repository.

Contents

I	Getting started	1
1	Introduction	2
1.1	What it does	2
1.2	Who it is for	2
1.3	Two packages	2
1.4	Three ideas behind the design	3
1.5	About this manual	3
2	Installation	4
2.1	Requirements	4
2.2	With conda or mamba	4
2.3	With pip and venv	5
2.4	The library alone, without Qt	5
2.5	Checking the installation	5
2.6	Platform notes	5
2.7	Building a standalone bundle	6
3	Quick start	7
3.1	Your first figure, from the command line	7
3.2	The same thing in the application	7
3.3	A publication figure	7
3.4	From Python	9
3.5	Where to go next	9
II	Working with trees	12
4	The interface	13
4.1	Layout of the window	13
4.2	The canvas	13
4.3	The command palette	14
4.4	Undo	14
4.5	Themes	14
4.6	Diagnostics	14
5	Tree file formats	15
5.1	What MakeYourTree reads	15
5.2	Writing	15
5.3	Lenient parsing	15

5.4	Security	16
5.5	Large files	16
6	Layouts	17
6.1	Branch modes	17
6.2	Rectangular	17
6.3	Slanted	18
6.4	Circular	18
6.5	Radial	19
6.6	Unrooted	19
6.7	Spacing and labels	23
7	Editing the tree	25
7.1	Rooting	25
7.2	Ordering	26
7.3	Collapsing	26
7.4	Pruning and extracting	27
7.5	Selecting	27
III	Annotation	28
8	The annotation format	29
8.1	A minimal file	29
8.2	Sections	30
8.3	The data section	30
8.4	Importing a spreadsheet	30
8.5	Leniency and round-tripping	31
8.6	Colours	31
9	Track reference	32
9.1	Options common to all tracks	32
9.2	Categorical tracks	32
9.3	Continuous tracks	34
9.4	Chart tracks	35
9.5	Structural tracks	37
9.6	Stacking tracks	39
IV	Output and automation	40
10	Exporting and projects	41
10.1	Exporting a figure	41
10.2	Sizes and resolution	41
10.3	What you see is what you get	41
10.4	Projects	42
10.5	Preparing a figure for publication	42
11	The command line	43
11.1	<code>info</code> — summarise a tree	43
11.2	<code>convert</code> — change format	43
11.3	<code>render</code> — write a figure	43

11.4	<code>reroot</code> — place a new root	45
11.5	Scripting	46
12	The Python API	47
12.1	Loading and inspecting	47
12.2	Traversal	47
12.3	Operations	48
12.4	Composing a figure	48
12.5	Attaching tracks	49
12.6	Raster and PDF output	49
12.7	Saving projects	49
12.8	A complete example	50
V	Appendices	51
A	Keyboard shortcuts	52
A.1	File	52
A.2	Edit and selection	52
A.3	Tree operations	53
A.4	Layout	53
A.5	View	53
A.6	Panels	54
A.7	Annotation and help	54
B	Troubleshooting	55
B.1	Installation	55
B.2	Files and parsing	55
B.3	Figures	56
B.4	Performance	56
B.5	Reporting a problem	56

Part I

Getting started

CHAPTER 1

Introduction

MakeYourTree is a desktop application and Python library for visualising and annotating phylogenetic trees. It runs entirely on your own machine: there is no server, no account, no upload and no network access at any point while it runs.

1.1 What it does

Reads what you already have.

Newick, New Hampshire eXtended (NHX), NEXUS and phyloXML. Parsing is deliberately forgiving — real files break their specifications constantly, so MakeYourTree loads them anyway and tells you what it had to interpret.

Draws it many ways.

Rectangular and slanted phylograms and cladograms, circular fans with a configurable arc and inner radius, radial layouts, and unrooted layouts by Felsenstein's equal-angle and equal-daylight algorithms.

Lets you edit the tree.

Reroot on any branch, by outgroup or at the midpoint; ladderize, rotate, collapse, prune and extract subtrees. Everything is undoable.

Annotates it.

Thirteen track types stack alongside the tree, described in [Chapter 9](#).

Exports honestly.

SVG and PDF as vectors, PNG at any resolution. The on-screen canvas and the exported file are built from an identical description, so what you export is what you saw.

1.2 Who it is for

Anyone who has a tree and needs a figure of it: for a paper, a thesis, a talk or a teaching handout. It is aimed particularly at people who cannot or would rather not upload their data to a hosted service — unpublished phylogenies, clinical isolate data, or work under a data-transfer agreement. Nothing leaves your machine, because the software has no way to send it anywhere.

The library half is aimed at a different need: regenerating a figure reproducibly, from a script, as part of an analysis pipeline. Anything you can build interactively you can also build in Python ([Chapter 12](#)) or from the command line ([Chapter 11](#)).

1.3 Two packages

MakeYourTree ships as two Python packages, and the split is architectural rather than legal — both are MIT-licensed.

Package	Contents
<code>makeyourtree</code>	Parsers, tree model, layout engine, annotation model, scene graph, SVG renderer and the command-line interface. Imports no GUI toolkit, so it works headlessly in scripts, notebooks and continuous integration. Depends only on NumPy.
<code>makeyourtree_studio</code>	The desktop application, built on PySide6/Qt 6.

If you only want to generate figures from a script, you never need the second package, and you can install without Qt entirely (Section 2.4).

1.4 Three ideas behind the design

You do not need this section to use MakeYourTree. It is here because it explains why certain things behave the way they do.

1.4.1 Two independent coordinates

Every layout is built from two coordinates that are computed separately. The *cross* coordinate — y in a rectangular layout, angle in a circular one — comes only from the order of the tips. The *along* coordinate — x , or radius — comes only from branch length, or from depth in a cladogram.

The practical consequence is that the two things you do to a figure never interfere. Ladderizing or rotating a clade changes the ordering and cannot alter any distance; switching between a phylogram and a cladogram changes the distances and cannot alter the ordering.

1.4.2 Band space: why one track works in every layout

Annotation tracks are not written for a particular layout. Each is written in *band space*, a coordinate system of (`row`, `offset`) where `row` indexes the tip and `offset` is the distance outward from the tree. A projector then maps band space into the page: a straight-line mapping for linear layouts, a polar one for circular layouts.

This is why a heatmap becomes an annulus in a circular layout without you doing anything, and why tangential labels stay the right way up on the left-hand side of a fan. Figures 3.2 and 3.3 are the same tracks with the same data; only the layout mode differs.

1.4.3 One scene, many outputs

Laying out and composing a figure produces a plain description of shapes and text. The on-screen canvas draws that description, and so do the SVG, PDF and PNG exporters. Because there is only one description, the screen and the file cannot disagree.

1.5 About this manual

Chapters 2–3 get you running. Chapters 4–7 cover the application. Chapters 8–9 cover annotation, and Chapter 9 is the reference for all thirteen track types. Chapters 10–12 cover output and automation.

Every figure in this manual was produced by MakeYourTree from the synthetic example data in the repository. Nothing is a mock-up.

CHAPTER 2

Installation

MakeYourTree runs on Windows, macOS and Linux from a single environment definition. The commands are the same on all three; the environment resolves the platform differences, including the Qt system libraries that would otherwise have to be installed by hand on Linux.

MakeYourTree is not on PyPI, so installation is from the source repository. There is no `pip install makeyourtree` from the index — ignore any package by that name from another author.

2.1 Requirements

- `git`
- A conda-family package manager, or Python 3.12 or newer for the pip route
- About 1 GB of disk space. Most of that is Qt: a complete environment measures 751 MB, of which PySide6 is 641 MB and NumPy 34 MB.

No network access is needed at runtime — only to install.

2.2 With conda or mamba

This is the recommended route. Substitute `mamba` or `micromamba` for `conda` throughout if you prefer; the arguments are identical and mamba resolves considerably faster.

```
git clone https://github.com/bioforkgithub/makeyourtree.git
cd makeyourtree
conda env create -f environment.yml
conda activate makeyourtree
pip install -e . --no-deps
```

-no-deps is not optional bookkeeping. Conda has already installed everything MakeYourTree needs. A plain `pip install -e .` would resolve those same requirements again from PyPI and install a *second* copy of Qt into an environment that already has one — 641 MB wasted, and two Qt builds present where which one loads depends on import order.

`Miniforge` is the usual recommendation for the package manager itself: it is small and defaults to the conda-forge channel, which is where `pyside6` comes from. On Anaconda or Miniconda the `defaults` channel takes priority and does not carry `pyside6`; if the environment fails to solve, prefer conda-forge:

```
conda config --add channels conda-forge
conda config --set channel_priority strict
```

2.3 With pip and venv

Use this if you would rather not install conda. It needs Python 3.12 or newer already present, and on Linux it needs the Qt system libraries installed by hand (Section 2.6.1).

```
git clone https://github.com/bioforkgithub/makeyourtree.git
cd makeyourtree
python -m venv .venv
```

Activate it. This is the only command in this manual that differs by platform:

Platform	Command
Linux, macOS	<code>source .venv/bin/activate</code>
Windows, PowerShell	<code>.venv\Scripts\Activate.ps1</code>
Windows, cmd.exe	<code>.venv\Scripts\activate.bat</code>

Then install. Note that here pip *does* resolve dependencies, so there is no `-no-deps`:

```
pip install -e ".[studio]"
```

2.4 The library alone, without Qt

The `makeyourtree` package and its command-line interface import no GUI toolkit, so they install without Qt. This is the right choice for a server, a container, a CI job or an analysis pipeline: it avoids the 641 MB PySide6 brings and needs none of the Linux system libraries below.

```
pip install -e "."
```

You then have the `makeyourtree` command and the Python API, but not `makeyourtree-studio`. NumPy is the only dependency. With conda, delete the `pyside6` line from `environment.yml` before creating the environment.

2.5 Checking the installation

```
makeyourtree --version
makeyourtree info examples/primates.nwk
```

To run the test suite as well — the fastest way to confirm a healthy install:

```
pip install -e ".[studio,dev]"
python -m pytest
```

Expect **2074 passed** in roughly 30 seconds. The suite runs headlessly and needs no display.

2.6 Platform notes

Two things genuinely differ by operating system. Both affect the pip route only; the conda environment handles the first and largely avoids the second.

2.6.1 Linux: Qt system libraries

The PySide6 *wheel* bundles Qt itself, but Qt still loads a few system libraries — above all its xcb platform plugin. Without them the application exits with `qt.qpa.plugin: Could not load the Qt platform plugin "xcb"`.

```
# Debian / Ubuntu
sudo apt install libgl1 libegl1 libglu1 libfontconfig1 libdbus-1-3 \
  libxkbcommon-x11-0 libxcb-cursor0 libxcb-icccm4 libxcb-image0 \
  libxcb-keysyms1 libxcb-randr0 libxcb-render-util0 libxcb-shape0 \
  libxcb-xinerama0

# Fedora
sudo dnf install mesa-libGL mesa-libEGL fontconfig dbus-libs \
  libxkbcommon-x11 xcb-util-cursor xcb-util-image xcb-util-keysyms \
  xcb-util-renderutil xcb-util-wm
```

`libxcb-cursor0` is required by Qt 6.5 and later specifically, and is the one most often missing on otherwise complete systems. On Debian and Ubuntu, `python3.12-venv` is also a separate package.

2.6.2 Windows: clone to a short path

PySide6 ships files whose names exceed Windows' 260-character `MAX_PATH` limit. Installing into a `.venv` inside a deeply nested directory fails with `OSError: [WinError 206] The filename or extension is too long`, and it rolls back *partially* — which is confusing, because the test suite still passes while `makeyourtree` itself is missing.

Clone somewhere short such as `C:\dev\makeyourtree`, or enable long paths with `git config --system core.longpaths true` together with the `LongPathsEnabled` registry setting. The conda route mostly sidesteps this, because the environment lives under the conda installation rather than inside the clone.

2.7 Building a standalone bundle

To produce a self-contained directory that runs without Python installed:

```
pip install pyinstaller
python -m makeyourtree_studio.packaging.build
```

The result is `dist/MakeYourTreeStudio/`. The build driver verifies the bundle against thirteen checks — licence texts present, Qt shipped as separate replaceable shared libraries, no GPL-3.0-only Qt add-on pulled in — and refuses to certify one that fails.

PyInstaller cannot cross-build: each platform's bundle must be produced on that platform. At the time of writing this has been built and launched on Windows only; macOS and Linux bundles are unproven rather than broken.

CHAPTER 3

Quick start

This chapter produces a finished, annotated figure in about five minutes. It uses the example data in the repository, so you can follow it exactly before substituting your own tree.

3.1 Your first figure, from the command line

The fastest route to a figure needs no interface at all:

```
makeyourtree render examples/primates.nwk figure.svg
```

That writes a rectangular phylogram as a vector SVG. Add a layout and an annotation track:

```
makeyourtree render examples/primates.nwk figure.svg \  
  --mode circular --align-tips \  
  --track examples/primates_region.mytrack
```

3.2 The same thing in the application

1. Start the application:

```
makeyourtree-studio
```

2. **File** → **Open** and choose `examples/primates.nwk`. The tree appears immediately.
3. Press **3** to switch to a circular layout, or **1** to go back to rectangular. The layout keys are **1–5** (Appendix A).
4. Press **A** to align the tips and draw guide lines.
5. **Annotate** → **Import Annotations** (**Ctrl+I**) and choose `examples/primates_region.mytrack`. A colour strip appears beside the tips, with a legend.
6. **File** → **Export Figure** (**Ctrl+E**). Choose SVG, PDF or PNG; the dialog predicts the pixel dimensions before you commit.

Every step is undoable with **Ctrl+Z**.

3.3 A publication figure

The example data includes a 24-tip bacterial tree with two annotation files. Loading the tree and both tracks gives Figure 3.2:


```
makeyoutree render examples/bacteria.nwk figure.svg --align-tips \
  --track examples/bacteria_resistance.mytrack \
  --track examples/bacteria_expression.mytrack
```

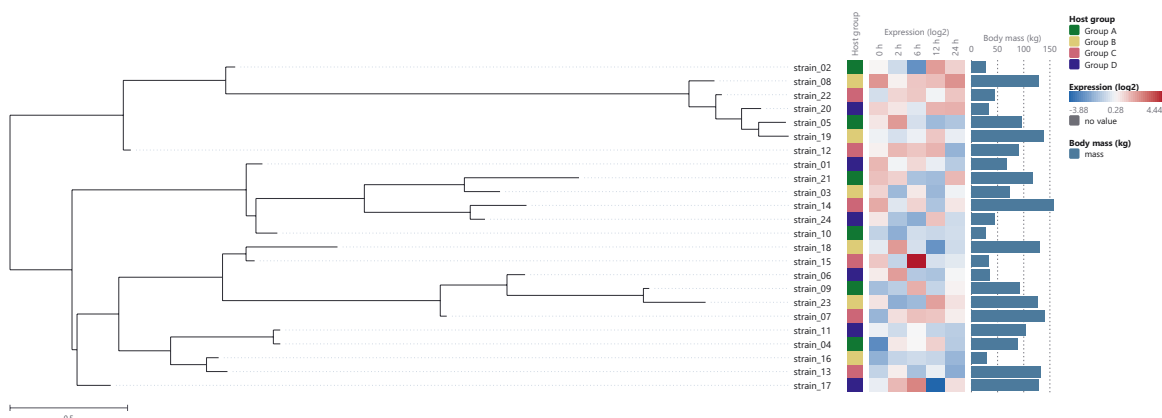


Figure 3.2. Three stacked tracks — a categorical colour strip, a diverging heatmap and a bar chart with a value axis — beside a rectangular phylogram. Tips are aligned and guide lines connect them to their labels.

Switching that same figure to a circular layout requires changing one option, and no track behaves differently:

3.4 From Python

The same figure, reproducibly, from a script:

```
from makeyoutree.io import load_tree
from makeyoutree.doc import Document
from makeyoutree.layout import LayoutParams, LayoutMode
from makeyoutree.scene import compose
from makeyoutree.render import render_svg
from makeyoutree.ops import midpoint_root, ladderize

tree = load_tree("examples/primates.nwk")
midpoint_root(tree).apply(tree)
ladderize(tree).apply(tree)

doc = Document(tree=tree,
               params=LayoutParams(mode=LayoutMode.CIRCULAR, arc=340))
with open("figure.svg", "w", encoding="utf-8") as fh:
    fh.write(render_svg(compose(doc)))
```

Chapter 12 covers the API in full.

3.5 Where to go next

- Chapter 5 — what MakeYourTree can read, and what it does with files that break their own specification.
- Chapter 6 — every layout mode, with figures.
- Chapter 8 — writing your own annotation files.

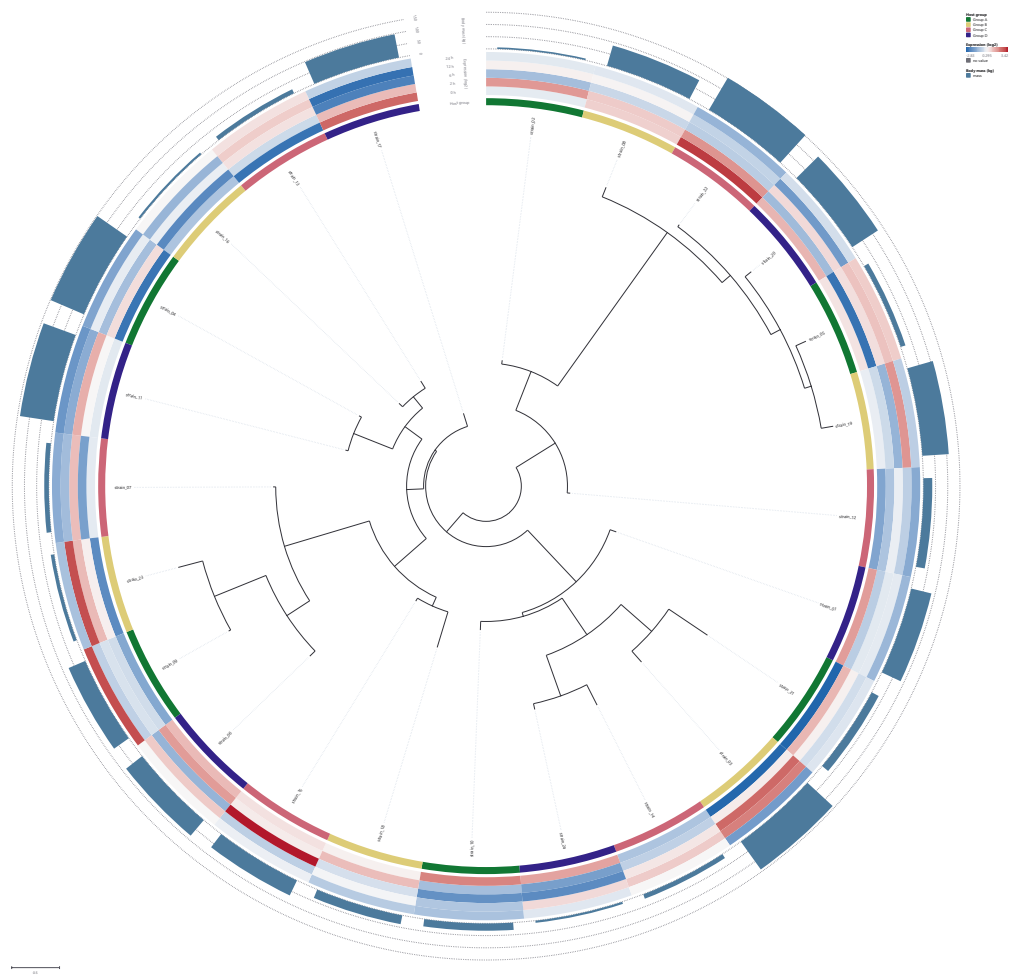


Figure 3.3. The identical tracks and data in a circular layout. The heatmap has become an annulus and the bars radiate outward; no track code differs between this figure and Figure 3.2. This is the band-space design of Section 1.4 doing its work.

- Chapter 9 — the reference for all thirteen track types.

Part II

Working with trees

CHAPTER 4

The interface

4.1 Layout of the window

The window is a canvas surrounded by dockable panels. Every panel can be shown or hidden from the **View** menu, and each has a shortcut (Appendix A).

Panel	Key	Purpose
Tree	Alt+1	The tree as a collapsible outline. Selecting a node here selects it on the canvas, and the reverse.
Style	Alt+2	Layout mode, branch mode, spacing, labels, theme. Most controls act immediately.
Tracks	Alt+3	The annotation stack. Reorder, hide, edit or remove tracks.
Search	Ctrl+F	Find taxa by name, with regular-expression support; matches are highlighted and selectable in bulk.
Inspector	Alt+4	Everything known about the current selection: name, branch length, support, depth and any parsed annotations.

4.2 The canvas

4.2.1 Navigating

- **Zoom** with the mouse wheel. Zoom is anchored at the pointer, and it never re-runs the layout — so zooming is instant regardless of tree size, and nothing shifts under the cursor.
- **Pan** by dragging with the middle button, or by dragging on empty canvas.
- **Ctrl+0** fits the whole figure to the window; **Ctrl+Shift+0** returns to actual size.

4.2.2 Selecting

Click a node to select it. Drag on empty canvas to rubber-band select a region. **Ctrl+Shift+A** extends the selection to the whole clade below the current node, **Ctrl+A** selects everything, and **Esc** clears the selection.

Selection drives the tree operations in Chapter 7: rerooting, collapsing and pruning all act on what is selected.

4.2.3 Context menu

Right-clicking a node offers the operations that apply to it — reroot here, collapse, extract subtree, remove — without going to the menu bar.

4.3 The command palette

Ctrl+P opens a searchable list of every command in the application. It is usually the fastest way to reach something whose menu location you cannot remember, and it shows each command's keyboard shortcut alongside it, so it doubles as a way to learn them.

The palette, the menus and the toolbar are all views of one registry of actions. An action therefore cannot appear in one and be missing from another, and a shortcut you rebind changes everywhere at once.

4.4 Undo

Every edit goes through a single command pipeline, so everything is undoable with **Ctrl+Z** and redoable with **Ctrl+Shift+Z**. This includes operations performed from the panels, not only those from the canvas.

Dragging a slider coalesces into a single history entry rather than one per pixel of movement, so undo steps back over the whole drag.

4.5 Themes

Ctrl+T switches between the light and dark themes. The theme belongs to the figure, not merely to the screen: the composed scene carries its own background colour, so an export made under the dark theme is dark in the file too.

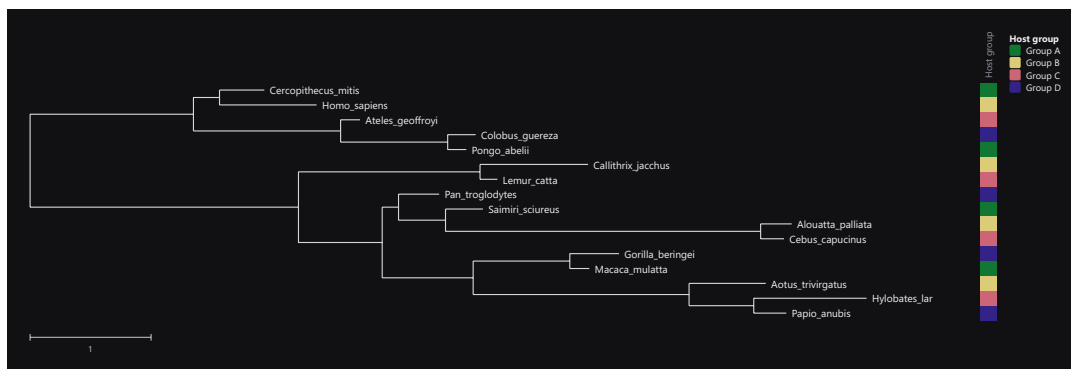


Figure 4.1. The dark theme. Exports match the screen because the background is part of the scene rather than a property of the window.

4.6 Diagnostics

When a file is loaded, anything MakeYourTree had to interpret is recorded as a diagnostic rather than discarded — a duplicate tip name, a negative branch length, an internal label that could be read either as a name or as a support value. The status bar reports how many there were, and **Ctrl+D** opens the full list.

These are informational far more often than they are errors. A clean file frequently produces one or two, and Chapter 5 explains the common ones.

CHAPTER 5

Tree file formats

5.1 What MakeYourTree reads

Format	Extensions	Notes
Newick	.nwk, .tree, .tre, .newick	Full grammar: single-quoted labels with doubled-quote escapes, nested comments, branch lengths, support values.
NHX	.nhx	New Hampshire eXtended. [&&NHX:key=value] annotations become typed node attributes.
NEXUS	.nex, .nexus	Including TRANSLATE blocks and [&R]/ [&U] rooting hints.
phyloXML	.xml, .phyloxml	Parsed with XML entity expansion disabled.

Format is detected from the content, not only the extension, so a Newick tree saved as `.txt` still loads. Pass `-format` to the CLI to override detection.

BEAST and FigTree [&key=value] metacommments are also parsed, so trees from those pipelines keep their posterior probabilities and rate annotations.

5.2 Writing

MakeYourTree writes Newick, NEXUS and phyloXML. NHX is read but not written as a separate format; NHX attributes are preserved on the nodes and can be written back into Newick comments.

5.3 Lenient parsing

Real-world tree files violate their specifications routinely: unbalanced quotes, duplicate tip labels, negative branch lengths, support values sitting where an internal label should be. A parser that refuses such files is a parser you cannot use on the files you actually have.

MakeYourTree therefore records each recoverable problem as a *diagnostic* and carries on. Exceptions are reserved for input that cannot be interpreted at all. Diagnostics are shown to you (`Ctrl+D`) rather than hidden, so leniency does not become silence.

5.3.1 Diagnostics you are likely to see

`newick.internal-label-ambiguous`

Internal labels that parse as numbers. These are almost always bootstrap or posterior support

values, so they are read as support. If yours are genuinely names, pass `internal_labels='name'` through the API. This is the single most common diagnostic and it is informational.

newick.duplicate-name

Two tips share a name. Both are kept; annotation matching by name will attach to both, which is usually not what you want.

newick.negative-length

A negative branch length. By default these are treated as zero for layout purposes, which is controlled by `ignore_negative_lengths`.

compose.approximate-band-space

Raised when annotation tracks are drawn in an unrooted layout. See the warning in Section 6.6.1.

5.4 Security

phyloXML is XML, and XML parsers are a classic vector for two attacks. MakeYourTree disables entity expansion entirely, which refuses both external-entity injection (reading files off your disk) and entity-expansion denial of service (the “billion laughs” attack). A phyloXML file from an untrusted source cannot use either against you.

5.5 Large files

Parsing is linear in tree size. A 100,000-tip Newick file parses in about 2.2 seconds and lays out in about 0.8 seconds on a normal workstation.

All traversals are iterative rather than recursive, which matters more than it sounds: a deeply unbalanced tree — the shape serially sampled sequence data produces — would exceed Python’s recursion limit at a few thousand nodes under a recursive implementation. MakeYourTree handles a 50,000-node caterpillar tree without difficulty.

The application remains responsive for trees of a few thousand tips. Beyond that, switching layout mode takes several seconds during which the window does not repaint; the work is not currently cancellable.

CHAPTER 6

Layouts

There are five layout modes, selected with keys 1–5, from the Style panel, or with `-mode` on the command line. Independently of the mode, the *branch mode* decides how the along-axis coordinate is derived.

6.1 Branch modes

Branch mode	Key	Along-axis coordinate
Phylogram	Ctrl+1	Cumulative branch length. Distances are meaningful.
Cladogram, aligned tips	Ctrl+2	Depth, with every tip on the outer edge. Topology only.
Cladogram, by level	Ctrl+3	Node depth in levels.

A phylogram shows evolutionary distance; a cladogram shows only branching order. Use a cladogram when branch lengths are absent or meaningless, and say which you used in the figure legend — readers cannot tell by looking.

6.2 Rectangular

The default, and the right choice for most figures. Branch length runs along *x*, tips are stacked down *y*.

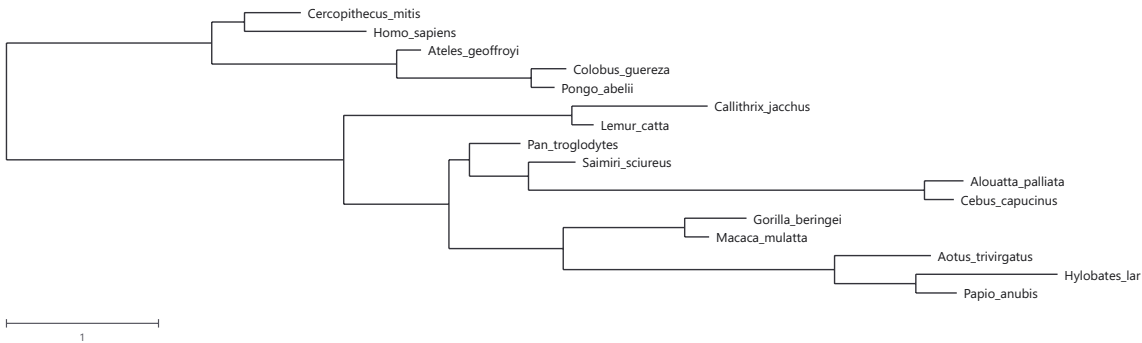


Figure 6.1. Rectangular phylogram. The scale bar at the lower left gives the units of the horizontal axis.

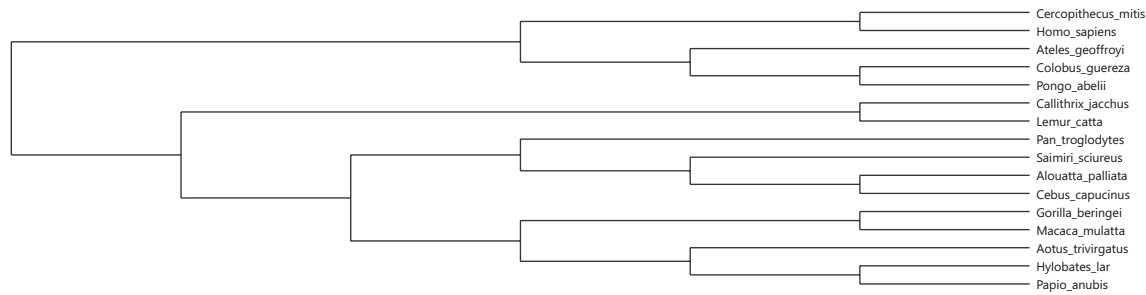


Figure 6.2. Rectangular cladogram with aligned tips. Horizontal position now carries no distance information.

6.3 Slanted

Branches are drawn as straight diagonals rather than as right angles. Compact and readable for small trees; harder to follow for large ones, because it is less obvious which ancestor a branch descends from.

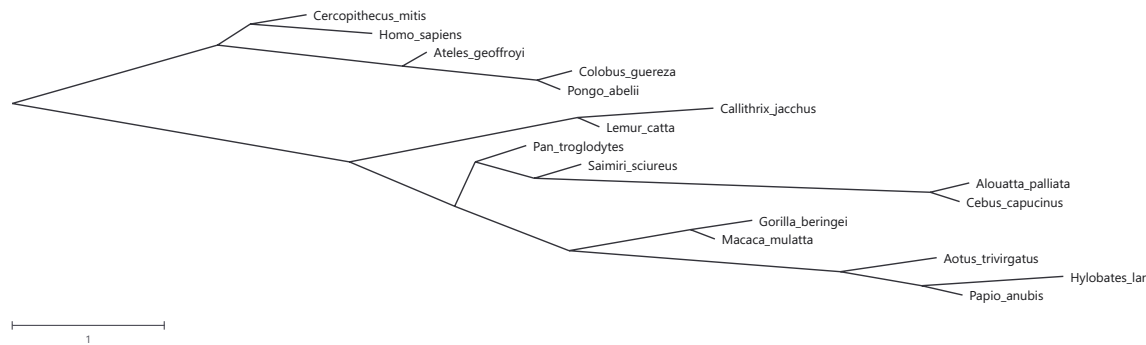


Figure 6.3. Slanted layout.

6.4 Circular

Tips are placed around an arc. This is the layout that makes a large tree fit on a page, and it is the natural home for multiple annotation rings.

Three options shape it:

Option	Default	Effect
arc	350°	Angular span. 360° closes the circle.
inner_radius	0.1	Hole at the centre, as a fraction of the radius.
start_angle	-90°	Where the first tip sits.

Leaving a small gap — the default 350° — makes it clear where the tip order begins and ends. Closing to a full circle looks tidier but loses that cue.

The gap does a second job that is easy to miss: it is the only place a track caption can go. A rectangular tree captions its columns in a header row above them and grows the page to fit; a fan has no room above its rings, so the captions are written into the wedge the arc leaves open.

Ask for several rings with long names and the wedge fills up, at which point MakeYourTree reports the captions it could not place rather than drawing them across the data. Widening the gap — `arc` at 340° or 320° — is the fix; the legend names every track either way.

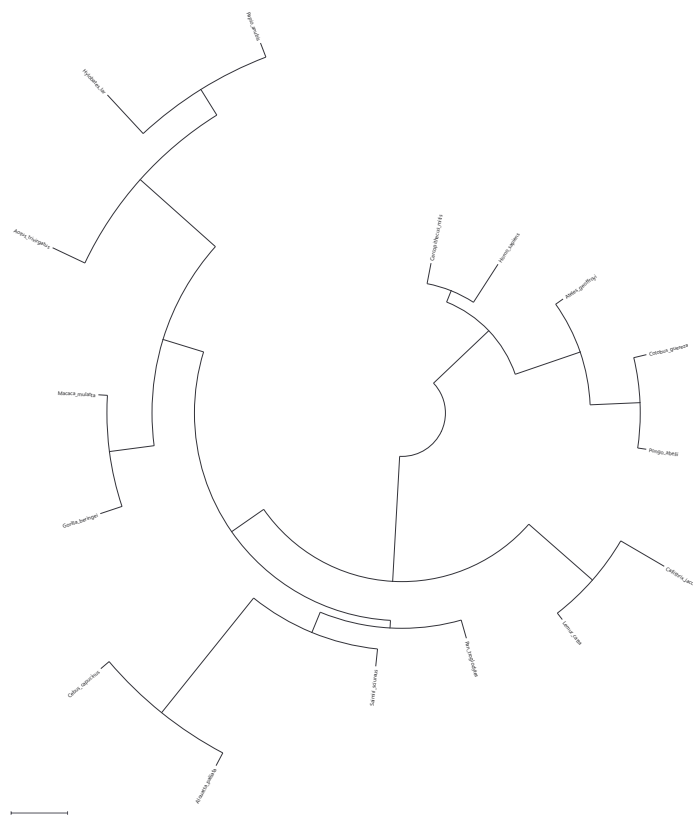


Figure 6.4. Circular fan with the default 350° arc. Labels are rotated tangentially and flipped on the left-hand side so that none of them is upside down.

A gap you should not try to fix. In a circular *phylogram*, tips sit at different radii because they are at different evolutionary distances from the root. A ragged inner edge against an annotation ring is therefore correct, not a bug. Turn on **Align tips** (A) if you want the ring flush.

6.5 Radial

Angle comes from tip order and radius from branch length, but the tree is drawn outward from a central root without the concentric-arc branch style of the circular mode.

6.6 Unrooted

An unrooted layout makes no claim about which node is ancestral. Two algorithms are available.

Equal-angle

Felsenstein's original method. Each subtree receives a wedge proportional to the number of tips it contains. Fast and always planar.

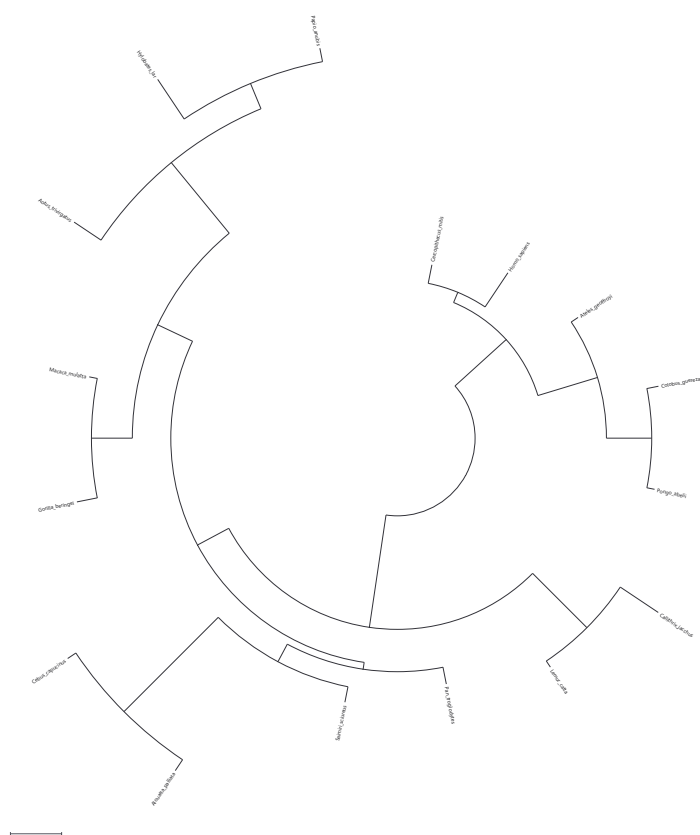


Figure 6.5. A full 360° circle with a larger inner radius.

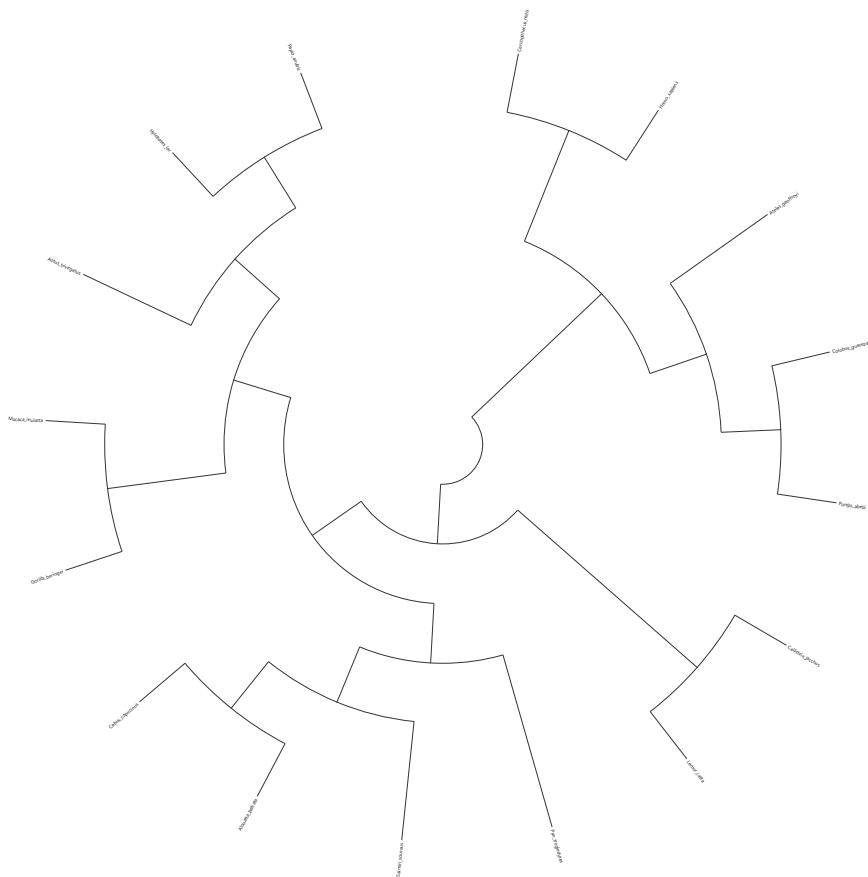


Figure 6.6. Radial layout.

Equal-daylight

Iteratively adjusts the wedges so the blank space between subtrees is evened out. Slower, usually easier to read.

Branch lengths are preserved exactly in both, and both are planar by construction.

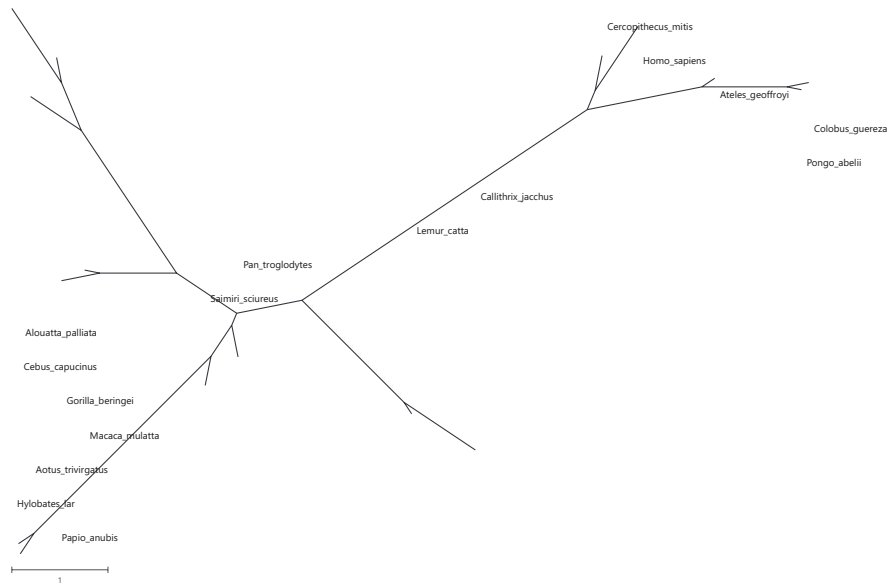


Figure 6.7. Unrooted, equal-angle.

6.6.1 Annotation tracks in unrooted layouts

Known limitation. An unrooted layout has no tip ordering along a line or a circle, so band space has no faithful meaning in it. Tracks are currently drawn as a column beside the figure, in tip order, *aligned to nothing*. The composition step emits the diagnostic `compose.approximate-band-space` to record this, but the interface does not yet present it prominently.

Do not use annotation tracks with an unrooted layout in a published figure without saying so. Use a rooted layout instead.

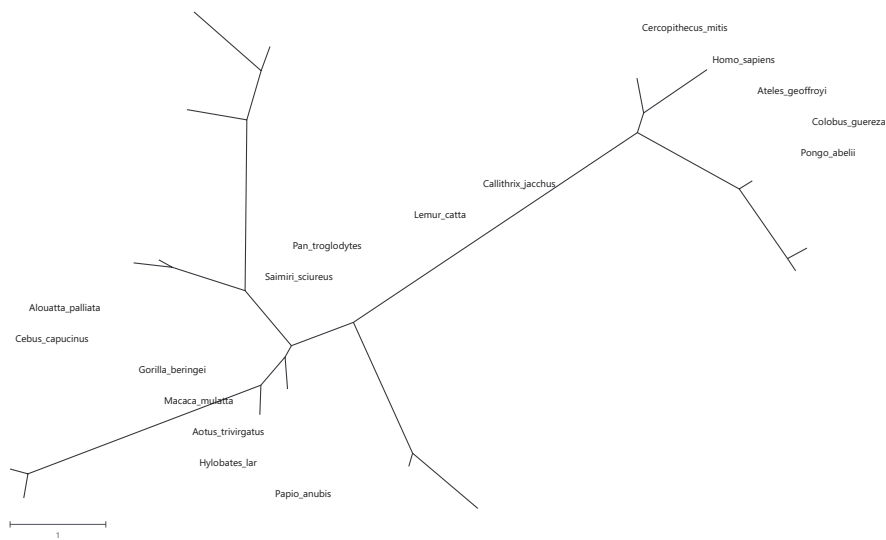


Figure 6.8. Unrooted, equal-daylight. The subtrees are spread to even out the gaps between them.

6.7 Spacing and labels

Setting	Effect
<code>row_spacing</code>	Distance between adjacent tip rows. The main control over figure height.
<code>width, height</code>	Page size in scene units.
<code>align_tips</code>	Flush all tip labels to a common edge.
<code>guide_lines</code>	Dotted leaders from each tip to its label. Only meaningful with aligned tips.
<code>show_tip_labels</code>	Turn tip labels off entirely — often necessary above a few hundred tips.
<code>max_label_width</code>	Truncate long labels with an ellipsis.

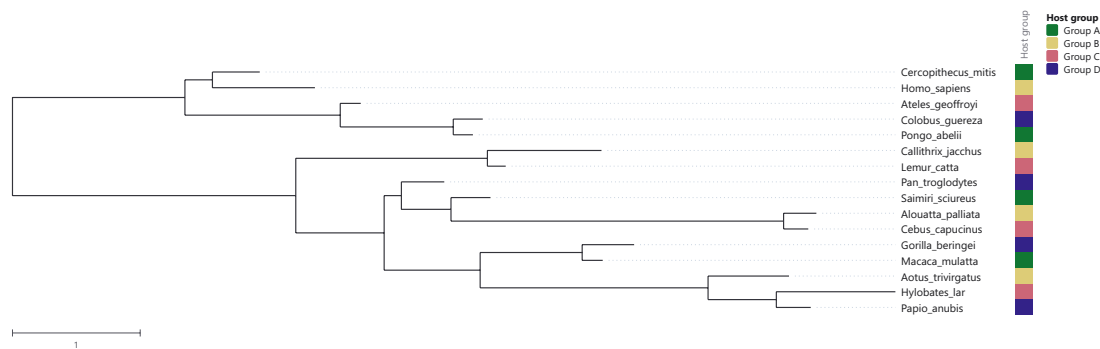


Figure 6.9. Tips aligned with guide lines. This is almost always what you want once an annotation track is attached, because it makes the rows line up visually with the track cells.

CHAPTER 7

Editing the tree

Every operation in this chapter is undoable (`Ctrl+Z`) and every one is also available from the Python API (Chapter 12).

7.1 Rooting

Where the root sits changes what the figure claims. It does not change the tree’s topology in the unrooted sense, but it entirely changes which groups look monophyletic, so it is worth being deliberate about.

Operation	Key	Description
Reroot here	R	Place the root on the branch above the selected node.
Midpoint root	M	Root at the midpoint of the longest tip-to-tip path.
Root on outgroup	Ctrl+Shift+R	Root against a named taxon or set of taxa.
Unroot	—	Remove the root, collapsing it into a trifurcation.

Midpoint rooting assumes a roughly constant rate of evolution across the tree. It is a reasonable default when you have no outgroup, and a poor one when rates vary substantially between lineages.

Outgroup rooting is preferable whenever you have a defensible outgroup, because it encodes an actual biological hypothesis rather than an assumption about rates.

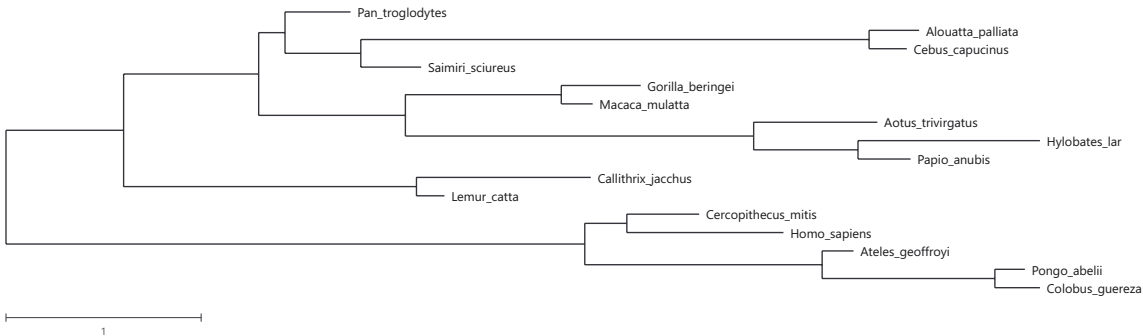


Figure 7.1. The example tree after midpoint rooting.

Branch lengths survive rerooting. A branch length belongs to the *edge* between a node and its parent, and rerooting reverses the direction of some edges. MakeYourTree carries the lengths across those reversals, and the test suite asserts that all pairwise tip-to-tip distances are unchanged by a reroot. Rerooting is a change of viewpoint, not of data.

7.2 Ordering

Reordering changes only the cross coordinate, so it can never alter a distance in the figure.

Operation	Key	Description
Ladderize ascending	L	Smaller subtrees first, giving the characteristic staircase.
Ladderize descending	Shift+L	Larger subtrees first.
Rotate children	Ctrl+R	Swap the order of the selected node's children.

Ladderizing is almost always worth doing before exporting: it makes the branching order far easier to follow, and costs nothing in accuracy.

7.3 Collapsing

Collapsing hides a clade's internal structure while keeping its position and extent, which is the standard way to show a large tree without printing every tip.

C collapses or expands the selected node; **Shift+C** expands everything. A collapsed clade is drawn as a triangle whose depth reflects the subtree it stands for.

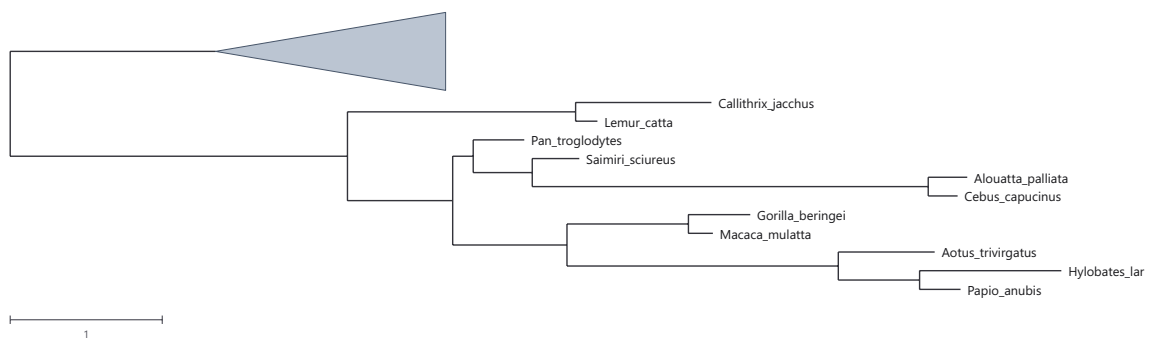


Figure 7.2. A collapsed clade. The triangle's size is derived from the hidden subtree rather than being fixed, so the figure still conveys how much was folded away.

The number of rows a collapsed clade occupies is configurable: `collapse_rows_mode` may be a square-root function of the subtree size (the default, which keeps large clades from dominating), a fixed number of rows, or proportional. `collapse_shape` selects the glyph.

7.4 Pruning and extracting

Operation	Key	Description
Remove selected taxa	<code>Del</code>	Delete tips and tidy away the internal nodes left dangling.
Extract subtree	<code>Ctrl+Shift+X</code>	Replace the document with the selected clade alone.

When pruning removes a tip, the internal node above it may be left with a single child. Such nodes are collapsed away, and the branch lengths on either side are *summed* rather than discarded, so the distances between the remaining tips are preserved.

7.5 Selecting

Editing acts on the selection, so the selection tools matter.

- Click a node, or drag a rubber band across several.
- `Ctrl+Shift+A` extends the selection to the entire clade below the current node.
- The Search panel (`Ctrl+F`) finds taxa by name or regular expression and can select every match at once — the practical way to select a scattered group.
- `Esc` clears the selection.

Part III

Annotation

CHAPTER 8

The annotation format

Annotation lives in `.mytrack` files. One file describes one track: its type, its data, and how it should look. The format is plain text and designed to be hand-editable, generated by a script, or exported from a spreadsheet.

8.1 A minimal file

```
#makeyourtree-annotation 1

[track]
type = color-strip
title = Biogeographic region

[columns]
names = region

[colors]
Neotropic = #117733
Afrotropic = #DDCC77
Palearctic = #332288

[legend]
title = Region
show = true

[data]
Homo_sapiens Palearctic
Pan_troglodytes Afrotropic
Aotus_trivirgatus Neotropic
```

The first line is the magic marker and is required. Everything after it is a series of `[section]` blocks.

8.2 Sections

Section	Contents
[track]	type (required), title, and match.
[options]	Appearance settings for this track type. Chapter 9 lists them per type.
[columns]	names = a, b, c — the column headers, used in legends and axis labels.
[colors]	Explicit colour per category, overriding the automatic palette.
[legend]	title, show, order.
[data]	One row per node: a key, then one value per column.

Only [track] with a type, and [data], are strictly required.

8.3 The data section

Each row begins with a key that identifies a node, followed by one value per column. Three separators are recognised, tried in this order:

1. a tab,
2. a comma,
3. a run of two or more spaces.

The last of these means a column-aligned table pasted out of a terminal works without editing. A single space never separates fields, so taxon names may contain spaces.

Quoting follows the CSV convention of RFC 4180: wrap a field in double quotes, and write a literal quote inside it twice.

8.3.1 Matching rows to nodes

By default keys are matched against tip names. `match = all` in the [track] section allows internal nodes to match too, which is what `clade-range` needs.

If the first data line begins with one of `id`, `key`, `name`, `node`, `label`, `taxon`, `tip` or `leaf`, it is treated as a header row rather than as data.

Rows that match nothing are not silently dropped: they are reported, and the application's import dialog shows a match report before you commit, so you find out about a naming mismatch before it becomes a wrong figure.

8.3.2 Missing is not zero

An empty field, or a single `-`, means *no value*. The track draws a gap there. This is deliberate and it matters: a zero would be a fabricated measurement, and a reader cannot tell a fabricated zero from a real one.

8.4 Importing a spreadsheet

You do not have to write `.mytrack` files by hand. **Annotate** → **Import Annotations** (**Ctrl+I**) reads CSV and tab-separated files directly. The wizard lets you choose the track type,

say which column holds the node key, and see how many rows matched before anything is added.

The usual workflow is to keep your data in a spreadsheet, export it as CSV, and import it. Save the resulting project (`.mytree`) and the track travels with it.

8.5 Leniency and round-tripping

Unknown sections and unknown keys are *kept*, not discarded, and recorded as diagnostics. If a future version writes a key this one does not understand, your file survives a load-and-save cycle unharmed.

A table written by `MakeYourTree` and read back is identical to the original. The writer quotes any field whose bare form would be re-read as something else — a number, a `-`, or a boolean.

The only genuinely fatal error is a file that declares no track type.

8.6 Colours

Colours may be given as `#RRGGBB`, `#RGB`, `rgb()` or `rgba()`. Bare colour *names* are deliberately not accepted in the `[colors]` section, because `red` is far more often a category label than a colour request — accepting it would silently merge a category with its own swatch.

When no explicit colours are given, categories are assigned from a colour-vision-safe palette. Twenty-one palettes are available, including the Paul Tol sets, Okabe–Ito, and the perceptually uniform `viridis` family. Prefer these over a hand-picked rainbow: roughly 1 in 12 men has a red–green colour vision deficiency, and a figure they cannot read is a figure that fails.

CHAPTER 9

Track reference

Thirteen track types are available. Each is described below with its data shape, its most useful options and a figure produced by MakeYourTree.

Every track works in every rooted layout mode without configuration, because tracks are written in band space (Section 1.4). The one exception is unrooted layouts, where alignment is not meaningful — see Section 6.6.1.

9.1 Options common to all tracks

Option	Default	Effect
<code>thickness</code>	varies	How much room the track takes, outward from the tree.
<code>gap_before</code>	—	Space between this track and the previous one.
<code>opacity</code>	1.0	Whole-track transparency.
<code>border_color</code> , <code>border_width</code>	—	Outline around drawn elements.
<code>show_title</code>	true	Draw the track title beside the stack.

Tracks are drawn in the order they appear in the Tracks panel, stacking outward. Drag to reorder.

9.2 Categorical tracks

9.2.1 Colour strip

One categorical value per tip, drawn as a band of colour. The most common annotation of all: host, region, lineage, clinical outcome.

Data: one column of category labels.

Key options: `colors` (explicit category colours), `palette`, `thickness`, `merge_runs`, `color_branches`, `color_labels`, `margin`.

`color_branches` and `color_labels` propagate the category colour onto the tree itself, which is often more readable than a separate band. `merge_runs` joins adjacent tips of the same category into one block.

9.2.2 Binary matrix

Presence and absence across several features — gene content, resistance profiles, trait matrices.

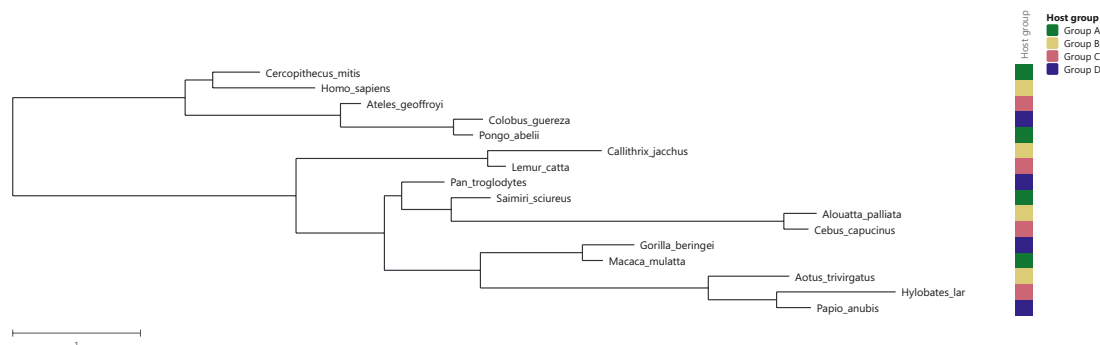


Figure 9.1. Colour strip with an explicit colour-vision-safe palette and a legend.

Data: one column per feature; values 1 (present), 0 (absent) and -1 or empty (unknown).
Key options: `shape` (square, circle, and others), `cell_size`, `column_colors`, `column_shapes`, `show_absent`, `label_rotation`.

Present, absent and unknown are three distinct states, and the track draws them differently. Do not encode unknown as absent.

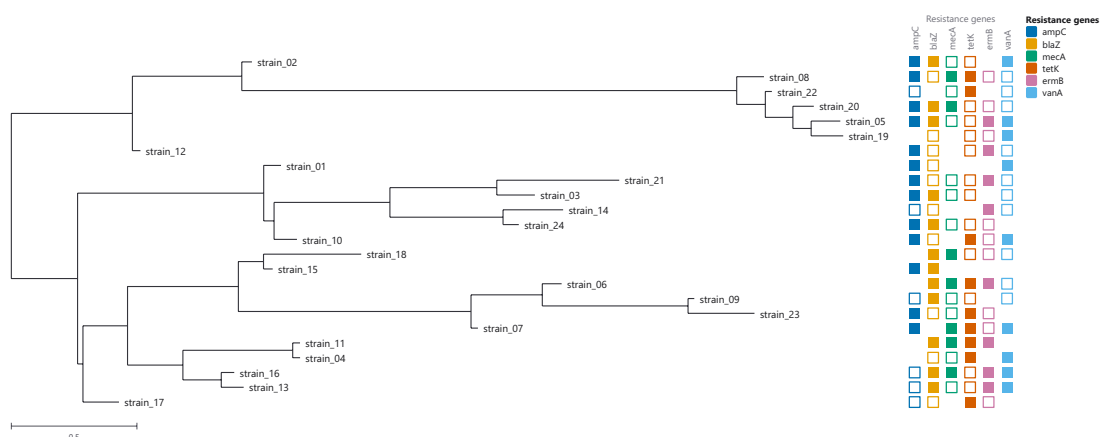


Figure 9.2. Binary matrix over six genes for 24 strains.

9.2.3 Symbols

A shape per node, its size and colour driven by data.

Data: typically one column for colour and one for size.

Key options: `color_column`, `size_column`, `shape`, `shape_column`, `size_mode`, `min_size`, `max_size`, `at_nodes`.

`size_mode` defaults to `sqrt`, so the drawn *area* is proportional to the value. This is the correct choice: readers judge a symbol by its area, and a radius-linear encoding exaggerates large values by roughly the square. `linear` is available for compatibility with tables authored against a radius-linear tool, but it is a decision you should make knowingly.

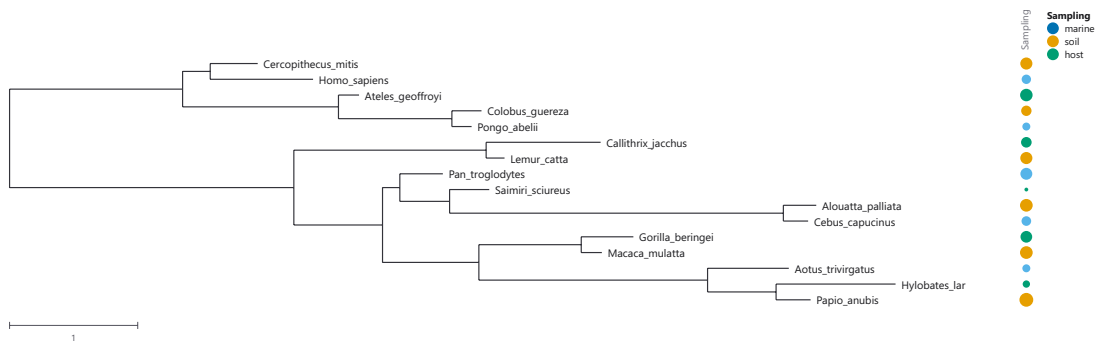


Figure 9.3. Symbols coloured by habitat and sized by count.

9.3 Continuous tracks

9.3.1 Gradient

A single continuous value per tip as one colour ramp. Compact — useful when you have many tracks and little room.

Data: one numeric column.

Key options: `color_min`, `color_mid`, `color_max`, `use_mid`, `min_value`, `max_value`, `stops`, `missing_color`.

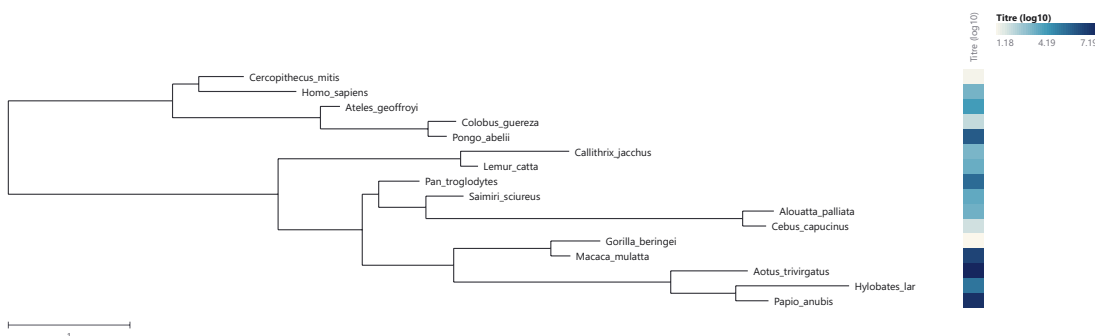


Figure 9.4. Gradient track.

9.3.2 Heatmap

A matrix of continuous values: expression across conditions, abundance across samples, distance to a reference.

Data: one column per condition.

Key options: `cell_size`, `cell_gap`, `normalize` (`global`, `row`, `column`), `color_min`, `color_mid`, `color_max`, `use_mid`, `cluster_columns`, `cluster_rows`, `show_values`, `column_labels`.

There is no palette option here — the ramp is set by `color_min`, `color_mid` and `color_max`. For data with a meaningful zero, such as log fold change, use a *diverging* ramp centred on white and set `use_mid`; a sequential ramp will hide the sign. `normalize` controls whether the scale is computed over the whole matrix, per row, or per column — and choosing wrongly is an easy way to mislead, so say which you used.

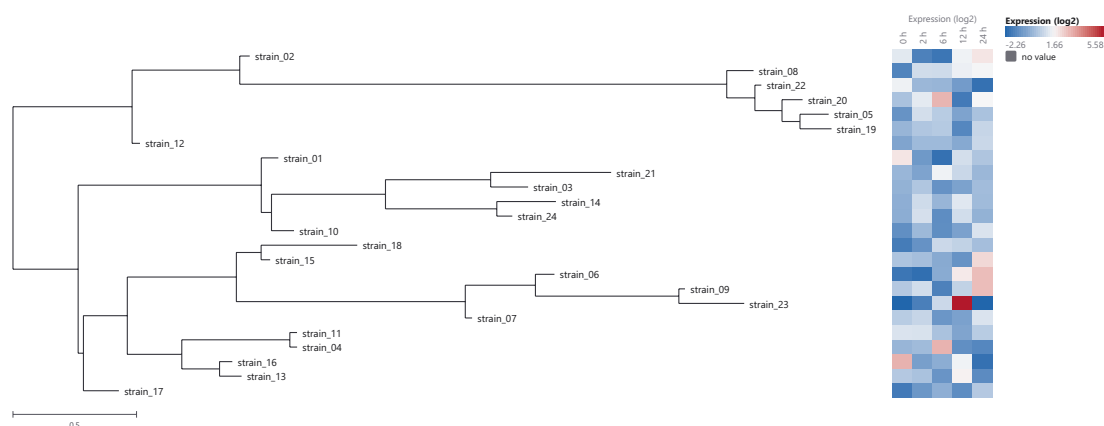


Figure 9.5. Heatmap with a diverging blue–white–red ramp, normalised globally.

9.4 Chart tracks

9.4.1 Bar chart

Data: one numeric column, or several for a stacked or grouped chart.

Key options: `stack`, `color`, `colors`, `axis`, `axis_ticks`, `value_min`, `value_max`, `zero_based`, `bar_gap`, `series_gap`.

Leave `zero_based` on unless you have a specific reason: a truncated bar axis misrepresents ratios, which is the most common way a bar chart lies.

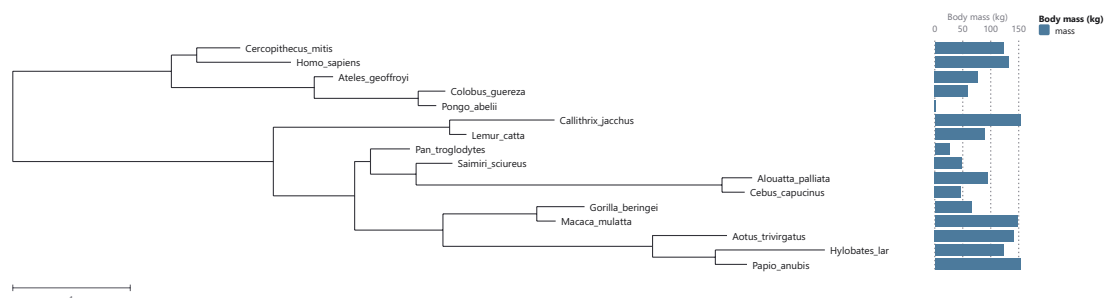


Figure 9.6. Bar chart with a value axis.

9.4.2 Box plot

Distribution per tip, rather than a single value.

Data: either raw replicate values (`source = raw`) or pre-computed five-number summaries.

Key options: `source`, `whisker_factor`, `show_outliers`, `box_height`, `box_color`, `median_color`, `axis`.

9.4.3 Pie chart

Part-to-whole composition at each node.

Data: one column per slice; values need not sum to 1.

Key options: `radius`, `donut`, `colors`, `size_column`, `at_nodes`, `start_angle`.

`donut` converts the pie to a ring, and `size_column` scales each pie by a further variable — useful for showing both composition and total.

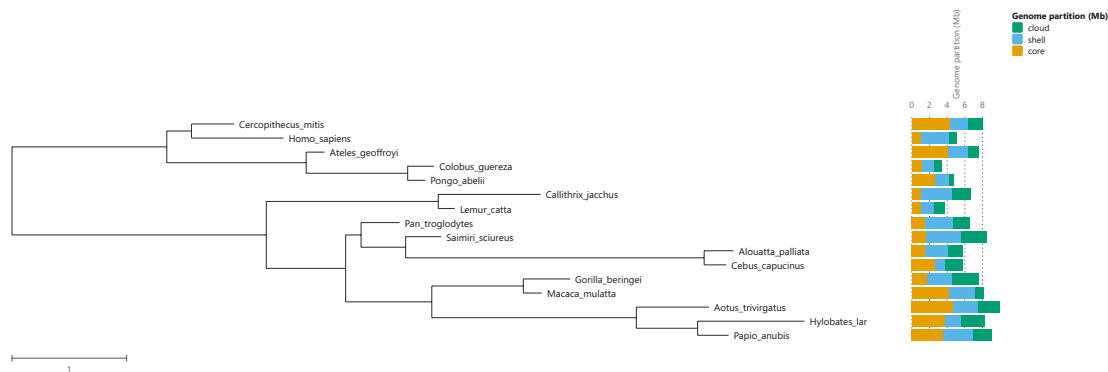


Figure 9.7. Stacked bar chart showing composition per tip.

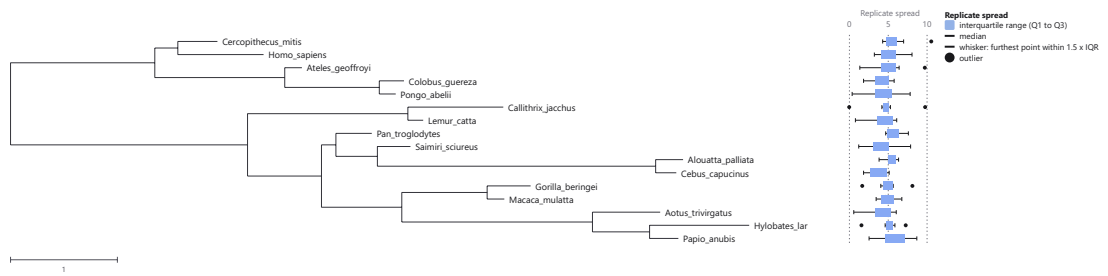


Figure 9.8. Box plots computed from seven raw replicates per tip.

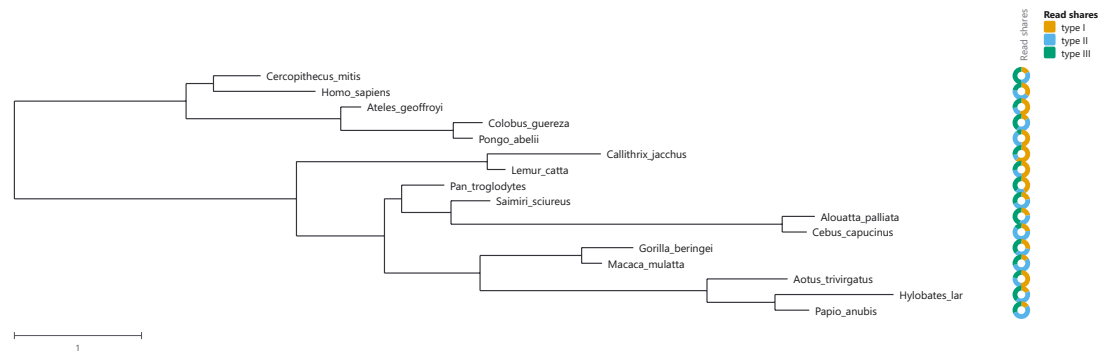


Figure 9.9. Donut charts at each tip.

9.4.4 Line chart

A small time series or profile per tip.

Data: one column per time point.

Key options: `zero_line`, `area`, `area_opacity`, `show_dots`, `dot_radius`, `line_width`, `y_range`, `value_min`, `value_max`.

Set `y_range` explicitly when comparing tips: with a per-row range, two rows of very different magnitude look identical.

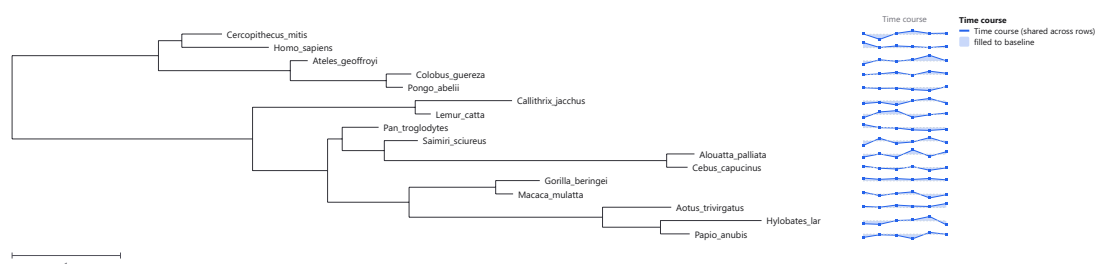


Figure 9.10. Line charts with a zero line and shaded area.

9.5 Structural tracks

9.5.1 Clade range

A shaded, labelled band behind an entire clade. The way to name and highlight groups.

Data: keyed by *internal* node, with a colour and a label. Requires `match = all`.

Key options: `show_labels`, `label_position`, `label_rotation`, `cover`, `extend`, `row_shrink`, `inner_offset`.

Drawn beneath the branches so the topology stays legible through the wash.

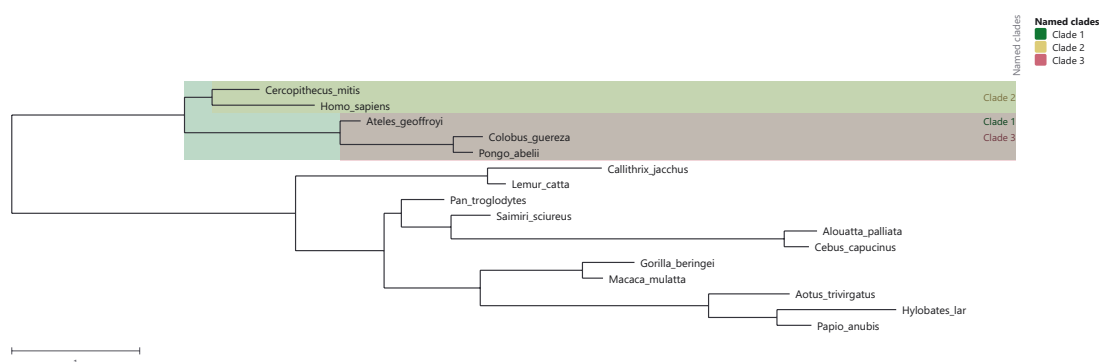


Figure 9.11. Clade ranges naming three groups.

9.5.2 Domain architecture

Protein features drawn to scale along a backbone — domains, motifs, transmembrane segments.

Data: a length column, then one column per feature, each formatted `start|end|shape|colour|label`.

Key options: `length_column`, `feature_height`, `backbone_height`, `show_labels`, `min_feature_width`.

Features are positioned in sequence coordinates, so proteins of different lengths are drawn on a common scale and are directly comparable.

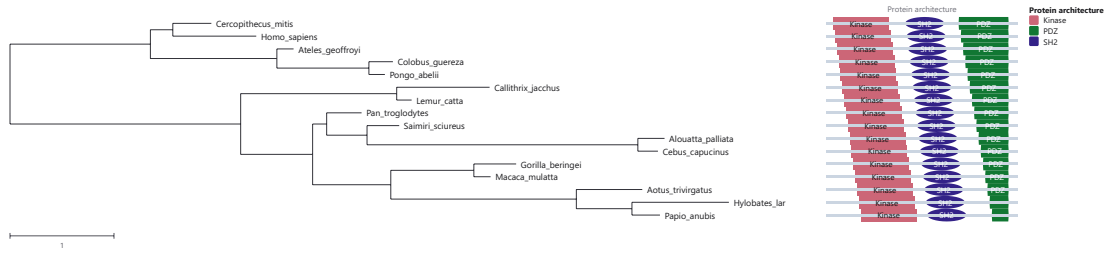


Figure 9.12. Domain architecture. Features are drawn at their true sequence positions along each backbone.

9.5.3 Text labels

Arbitrary text beside each tip, independent of the tip name.

Data: one column of strings.

Key options: size, color, color_column, align, bold, italic, max_width, ellipsize, rotate_with_layout.

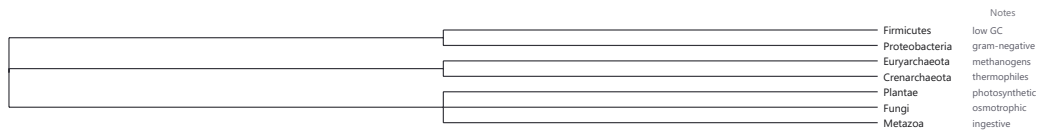


Figure 9.13. Text labels carrying notes independent of the tip names.

9.5.4 Connections

Curved links between arbitrary node pairs: gene transfer, recombination, host jumps, co-occurrence.

Data: each row gives two node names, and optionally a weight, a colour and a label.

Key options: bow, width_scale, min_width, max_width, show_labels, color, anchor.

This track is unusual in two ways: it annotates a *relation* rather than a node, and it consumes no thickness at all. Links are drawn over the tree body, beneath the branches, so a dense bundle never obscures the topology it is drawn against.

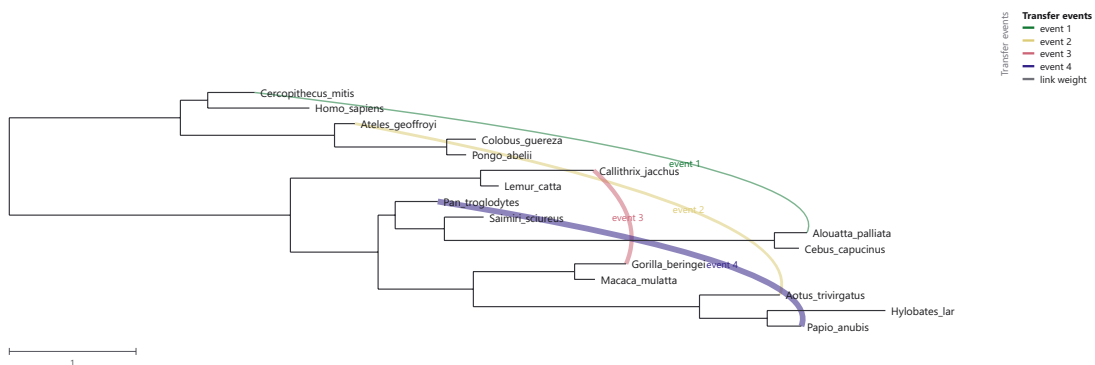


Figure 9.14. Connections between four pairs of tips, with width scaled by weight.

9.6 Stacking tracks

Tracks stack outward in panel order, and any combination may be used together. Figure 9.15 shows a categorical strip and a heatmap around a 24-tip tree.

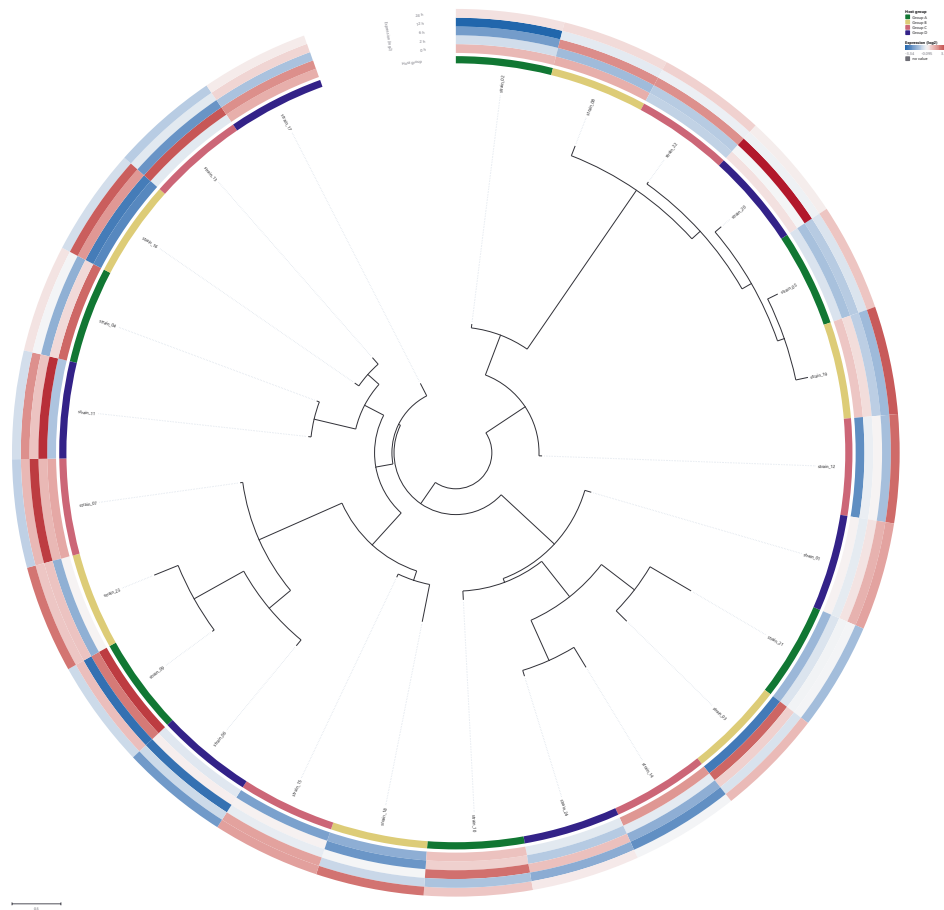


Figure 9.15. A colour strip and a heatmap wrapped around a circular layout with aligned tips.

Part IV

Output and automation

Exporting and projects

10.1 Exporting a figure

File → **Export Figure** (`Ctrl+E`) writes the current figure. Three formats are available.

Format	Kind	Use it for
SVG	Vector	Journal submission, and any figure you intend to edit afterwards in a vector editor.
PDF	Vector	Journal submission, and inclusion in $\text{\LaTeX}$ documents.
PNG	Raster	Slides, web pages, quick sharing.

Prefer a vector format wherever it is accepted. A vector figure has no resolution: it stays sharp at any magnification and in print. Use PNG only where raster is required.

For PNG, a scale factor multiplies the figure’s natural size. The export dialog predicts the resulting pixel dimensions before you commit, so you can hit a journal’s minimum resolution without guessing. A scale of 2–4 is typical for print.

10.2 Sizes and resolution

A scene unit is a **PostScript point**, $1/72$ inch, and every output honours that. An SVG carries `pt` on its `width` and `height`, a PDF page is measured in points, and a PNG multiplies by `dpi/72`. The consequence worth knowing is that the same figure exported as SVG and as PDF is the *same physical size*, so the two can be swapped in a manuscript without anything moving.

For raster output the DPI box and the pixel boxes are two views of one number: changing either updates the other. 300dpi is the usual journal minimum and 600dpi is sometimes required. The pixel size predicted in the dialog is exactly what lands on disk.

The command line offers the same controls, plus physical units and named page sizes — see Section 11.

10.3 What you see is what you get

The canvas and all three exporters consume an identical description of the figure. There is no separate rendering path for export, so a figure cannot differ between the screen and the file. This includes the theme: a figure composed under the dark theme exports dark.

10.4 Projects

A project file (`.mytree`) holds the tree, the layout settings, the theme and every annotation track, so reopening it restores the figure exactly as you left it.

Action	Key
Save	Ctrl+S
Save As	Ctrl+Shift+S
Open Project	Ctrl+Shift+O

A project is a container carrying a manifest plus the tree and annotation data. Because the tracks travel inside it, a project is self-contained: you can move it to another machine and it will open with its annotations intact, without the original `.mytrack` files alongside.

Projects are treated as untrusted input on load, in the same way tree files are: a project arriving by email or download is parsed defensively rather than trusted.

10.5 Preparing a figure for publication

A short checklist, in the order that tends to work:

1. **Root deliberately.** Outgroup rooting if you have an outgroup, midpoint otherwise — and say which in the caption (Chapter 7).
2. **Ladderize** (`L`). It costs nothing and makes branching order far easier to follow.
3. **Align tips** (`A`) once you have annotation tracks, so rows line up with track cells.
4. **Check the colours.** Prefer the built-in colour-vision-safe palettes over a hand-picked rainbow.
5. **Say what the branch lengths mean**, or turn them off with a cladogram if they are meaningless. Readers cannot tell by looking.
6. **State the support values** shown, and what they are — bootstrap percentages and posterior probabilities look identical and mean quite different things.
7. **Export vector** (SVG or PDF).
8. **Save the project** alongside the figure, so the figure can be regenerated or corrected later.

For a figure that must be reproducible from a pipeline rather than from a saved project, generate it from a script instead — Chapters 11 and 12.

CHAPTER 11

The command line

The `makeyourtree` command needs no GUI toolkit, so it works on a headless server, in a container and in continuous integration. It has four subcommands.

```
makeyourtree --help
makeyourtree <command> --help
```

11.1 info — summarise a tree

```
makeyourtree info examples/primates.nwk
```

Reports the detected format, tip and node counts, whether the tree is rooted and binary, its height, the longest root-to-tip distance, whether branch lengths and support values are present, and whether it is ultrametric. Any diagnostics from parsing are printed first.

Option	Effect
<code>--format</code>	Override format detection.
<code>--json</code>	Emit a JSON object instead of the text summary.

`-json` is the one to use from a script; the text layout is for humans and may change.

11.2 convert — change format

```
makeyourtree convert tree.nex tree.xml
makeyourtree convert tree.nwk out.nex --format nexus
```

The output format is inferred from the extension unless `-format` says otherwise. `-input-format` overrides detection on the input. Newick, NEXUS and phyloXML can be written; all four formats can be read.

11.3 render — write a figure

The subcommand that does the most work. It writes SVG, PDF or PNG.

```
makeyourtree render examples/primates.nwk figure.svg \
  --mode circular --branch-mode phylogram \
  --ladderize asc --align-tips \
```

```
--track examples/primates_region.mytrack \
--theme light --width 1200
```

Option	Effect
--mode	rectangular, slanted, circular, radial, unrooted.
--branch-mode	phylogram, cladogram-aligned, cladogram-level.
--width, --height	Page size, with an optional unit: 180mm, 7in, 900pt, 1200px. A bare number is points.
--output-format	svg, pdf or png. Defaults to whatever the output extension says.
--dpi	Resolution for raster output, default 300. Ignored by the vector formats.
--page	A named page size instead of --width/--height.
--landscape	Swap the axes of --page.
--row-spacing	Distance between adjacent tip rows.
--ladderize	asc or desc, applied before layout.
--align-tips	Flush tip labels and draw guide lines.
--no-tip-labels	Omit tip labels entirely.
--track FILE	Attach an annotation file. Repeatable; tracks stack in the order given.
--theme	light or dark.
--format	Override format detection on the input.

11.3.1 Choosing the format

The output extension picks the format, so the common case needs no flag:

```
makeyourtree render tree.nwk figure.svg # vector
makeyourtree render tree.nwk figure.pdf # vector
makeyourtree render tree.nwk figure.png # raster, 300 dpi
```

--output-format overrides it when the extension is missing or misleading.

PNG and PDF need the desktop extra. Rasterising requires Qt, and the `makeyourtree` library deliberately does not depend on it. With the library alone SVG works, and asking for a PNG tells you to run `pip install 'makeyourtree[studio]'` rather than failing with a traceback. SVG needs nothing beyond the library and is a fine choice for a headless pipeline.

11.3.2 Resolution

--dpi sets pixels per inch for raster output. The default of 300 is the usual journal minimum; 600 is asked for by some, and 150 is plenty for a slide.

```
makeyourtree render tree.nwk figure.png --dpi 600
```

It does nothing to SVG or PDF, which have no pixels. A vector figure is sharp at any size, which is why it is the better choice wherever the destination takes one.

11.3.3 Physical size

`-width` and `-height` take a unit — mm, cm, in, pt or px. A bare number is points, so an existing `-width 900` keeps its old meaning.

```
makeyourtree render tree.nwk figure.pdf --width 180mm
makeyourtree render tree.nwk figure.png --width 7in --dpi 600
makeyourtree render tree.nwk figure.pdf --page a4 --landscape
```

Preset	Size
a5, a4, a3	ISO paper, portrait
letter, legal, tabloid	US paper, portrait
column	A single journal column; height left to the figure
wide-column	A double column; height left to the figure

An explicit `-width` or `-height` overrides the preset on that axis, so a preset can serve as a starting point.

A requested size is a floor, never a crop. If the figure needs more room than you asked for — long tip labels are the usual reason — it is drawn at its natural size and nothing is cut off. The command says so on standard error rather than leaving you to discover it later. To make a figure genuinely fit, shorten or drop the tip labels, reduce `-row-spacing`, or scale it down when you place it.

Every run reports what it produced, so the size is discoverable without opening the file:

```
$ makeyourtree render examples/primates.nwk figure.png --dpi 300
figure.png 4264 x 1410 px at 300 dpi (361.0mm x 119.3mm)
```

`-q` before the subcommand silences it.

`-track` may be given more than once, which is how you build a multi-track figure non-interactively:

```
makeyourtree render examples/bacteria.nwk figure.svg --align-tips \
--track examples/bacteria_resistance.mytrack \
--track examples/bacteria_expression.mytrack
```

11.4 reroot — place a new root

```
makeyourtree reroot tree.nwk rooted.nwk --midpoint
makeyourtree reroot tree.nwk rooted.nwk --outgroup Homo_sapiens,Pan_troglodytes
```

Exactly one of `-midpoint` or `-outgroup` is required. `-outgroup` takes a comma-separated list of taxon names. The output format follows the extension unless `-output-format` says otherwise.

Pairwise tip-to-tip distances are unchanged by rerooting.

11.5 Scripting

The commands compose in a pipeline. To convert, reroot and render a directory of trees:

```
for f in trees/*.nwk; do
    base=$(basename "$f" .nwk)
    makeyourtree reroot "$f" "rooted/$base.nwk" --midpoint
    makeyourtree render "rooted/$base.nwk" "figures/$base.svg" \
        --mode circular --ladderize asc --align-tips
done
```

Every subcommand exits non-zero on failure, so `set -e` behaves as expected.

For anything more elaborate — conditional styling, computed annotations, tracks built from data you already have in memory — use the Python API in Chapter 12 rather than shelling out repeatedly.

CHAPTER 12

The Python API

The `makeyourtree` package imports no GUI toolkit, so everything in this chapter runs in a script, a notebook, a container or a CI job.

12.1 Loading and inspecting

```
from makeyourtree.io import load_tree, save_tree

tree = load_tree("examples/primates.nwk")

print(len(tree.nodes))
print([n.name for n in tree.nodes if not n.children])
print(tree.rooted)
```

`load_tree` detects the format from the content. Pass `format=` to override it. `load_trees` reads a multi-tree file.

12.1.1 Diagnostics

Parsing problems are collected rather than raised. To see them, pass a sink:

```
from makeyourtree.core.diagnostics import DiagnosticSink

sink = DiagnosticSink()
tree = load_tree("messy.nwk", sink=sink)
for d in sink:
    print(d.code, d.message)
```

12.2 Traversal

Never recurse over a tree. A deeply unbalanced tree of 100,000 tips has a path length of 100,000, and Python's recursion limit is 1,000. Use the iterative traversals in `makeyourtree.core.traversal`; every traversal inside `MakeYourTree` is iterative for exactly this reason.

```
from makeyourtree.core import traversal

for node in traversal.preorder(tree.root):
    ...
for node in traversal.postorder(tree.root):
```

...

12.3 Operations

Operations return a *command* rather than mutating immediately, which is what makes them undoable in the application. From a script, apply it at once:

```
from makeyourtree.ops import (midpoint_root, outgroup_root, ladderize,
                             rotate, collapse, prune, extract_subtree, unroot)

midpoint_root(tree).apply(tree)
ladderize(tree, ascending=True).apply(tree)
```

To keep an undo history, use a `CommandStack`:

```
from makeyourtree.ops import CommandStack

stack = CommandStack()
stack.do(midpoint_root(tree), tree)
stack.do(ladderize(tree), tree)
stack.undo(tree) # back to midpoint-rooted only
```

A branch length belongs to an edge, not to a node. It is stored on the child because that is where a parent-pointer tree can hold it in constant time, but it describes the edge *parent* $\rightarrow$ *child*. Any operation that reverses an edge — rerooting, unrooting, grafting — must carry the length and support across the reversal. MakeYourTree does; if you write your own tree surgery, this is the invariant to test, and testing that pairwise patristic distances survive a reroot is the way to catch a breach.

12.4 Composing a figure

```
from makeyourtree.doc import Document
from makeyourtree.layout import LayoutParams, LayoutMode, BranchMode
from makeyourtree.scene import compose
from makeyourtree.render import render_svg
from makeyourtree.style import LIGHT, DARK

doc = Document(
    tree=tree,
    params=LayoutParams(mode=LayoutMode.CIRCULAR, arc=340.0,
                        align_tips=True, row_spacing=16.0),
    theme=LIGHT,
)

scene = compose(doc)
with open("figure.svg", "w", encoding="utf-8") as fh:
    fh.write(render_svg(scene))
```

`compose` returns a `Scene`: plain data describing layered shapes and text, with no reference to any rendering technology. `scene.count()` gives the number of marks, and `scene.iter_marks()` walks them in painting order.

12.5 Attaching tracks

From a file:

```
from makeyourtree.annot import load_annotation

track = load_annotation("examples/primates_region.mytrack", tree)
doc.tracks.append(track)
```

Or built directly, which is the point of the API — annotation computed from data you already have, without writing a file:

```
from makeyourtree.tracks.strip import ColorStripTrack

track = ColorStripTrack(
    title="Region",
    options={"thickness": 18,
            "colors": {"Africa": "#117733", "Asia": "#332288"}},
)
track.bind(tree, {"Homo_sapiens": ["Africa"],
                  "Macaca_mulatta": ["Asia"]}, columns=["region"])
doc.tracks.append(track)
```

`bind` takes a mapping of node name to a list of values, one per column, and resolves the names against the tree. Chapter 9 lists the class and options for each track type.

12.6 Raster and PDF output

The core writes SVG. PNG and PDF come from the studio package, which requires Qt and a `QApplication`:

```
from PySide6.QtWidgets import QApplication
from makeyourtree_studio.export.raster import export_png, export_pdf

app = QApplication.instance() or QApplication([])
export_pdf(scene, "figure.pdf")
export_png(scene, "figure.png", scale=3.0)
```

Fonts and the offscreen platform. Qt's offscreen platform reports zero font families, so text rasterises as empty boxes. If you are rendering PNGs in a headless job and the text is missing, that is the cause. Clear `QT_QPA_PLATFORM` before importing Qt, or use a virtual framebuffer such as `xvfb-run`. SVG output from the core is unaffected, because it embeds text as text.

12.7 Saving projects

```
from makeyourtree.doc.io import save_document, load_document

save_document(doc, "figure.mytree")
doc2 = load_document("figure.mytree")
```

Do not put live objects in `document.metadata`. It is serialised on save, and an unserialisable value there will fail the write. Runtime-only keys are stripped on save; anything else you add must be JSON-representable.

12.8 A complete example

```
from makeyourtree.io import load_tree
from makeyourtree.doc import Document
from makeyourtree.layout import LayoutParams, LayoutMode
from makeyourtree.ops import midpoint_root, ladderize
from makeyourtree.scene import compose
from makeyourtree.render import render_svg
from makeyourtree.annot import load_annotation

tree = load_tree("examples/bacteria.nwk")
midpoint_root(tree).apply(tree)
ladderize(tree).apply(tree)

doc = Document(tree=tree,
               params=LayoutParams(mode=LayoutMode.CIRCULAR,
                                   arc=340.0, align_tips=True))
for path in ("examples/bacteria_resistance.mytrack",
             "examples/bacteria_expression.mytrack"):
    doc.tracks.append(load_annotation(path, tree))

with open("figure.svg", "w", encoding="utf-8") as fh:
    fh.write(render_svg(compose(doc)))
```

`tools/make_gallery.py` in the repository is a longer worked example: it builds every figure in this manual, exercising all thirteen track types and every layout mode.

Part V

Appendices

APPENDIX A

Keyboard shortcuts

Every shortcut is rebindable: **Help** → **Keyboard Shortcuts** (`Ctrl+K`). The command palette (`Ctrl+P`) shows each command's current binding alongside it.

On macOS, read `Ctrl` as `Cmd`.

A.1 File

Action	Shortcut
Open Project...	<code>Ctrl+Shift+O</code>
Save	<code>Ctrl+S</code>
Save As...	<code>Ctrl+Shift+S</code>
Export Figure...	<code>Ctrl+E</code>
Quit	<code>Ctrl+Q</code>

A.2 Edit and selection

Action	Shortcut
Undo	<code>Ctrl+Z</code>
Redo	<code>Ctrl+Shift+Z</code>
Select All	<code>Ctrl+A</code>
Select Clade	<code>Ctrl+Shift+A</code>
Clear Selection	<code>Esc</code>

A.3 Tree operations

Action	Shortcut
Reroot Here	R
Midpoint Root	M
Root on Outgroup	Ctrl+Shift+R
Ladderize Ascending	L
Ladderize Descending	Shift+L
Rotate Children	Ctrl+R
Collapse / Expand	C
Expand All	Shift+C
Remove Selected Taxa	Del
Extract Subtree	Ctrl+Shift+X

A.4 Layout

Action	Shortcut
Rectangular	1
Slanted	2
Circular	3
Radial	4
Unrooted	5
Phylogram	Ctrl+1
Cladogram (aligned tips)	Ctrl+2
Cladogram (by level)	Ctrl+3
Align Tips	A

A.5 View

Action	Shortcut
Zoom In	Ctrl+=
Zoom Out	Ctrl+-
Fit to Window	Ctrl+0
Actual Size	Ctrl+Shift+0
Dark Theme	Ctrl+T
Command Palette...	Ctrl+P

A.6 Panels

Panel	Shortcut
Tree Panel	Alt+1
Style Panel	Alt+2
Tracks Panel	Alt+3
Search Panel	Ctrl+F
Inspector Panel	Alt+4

A.7 Annotation and help

Action	Shortcut
Import Annotations...	Ctrl+I
Add Track...	Ctrl+Shift+T
Manage Tracks	Ctrl+Shift+M
Keyboard Shortcuts...	Ctrl+K
Diagnostics...	Ctrl+D

APPENDIX B

Troubleshooting

B.1 Installation

Could not load the Qt platform plugin "xcb" (Linux)

Missing Qt system libraries. Install them (Section 2.6.1), or use the conda environment, which supplies them as packages. To see exactly which library is missing, run `QT_DEBUG_PLUGINS=1 makeyourtree-studio`.

OSError: [WinError 206] The filename or extension is too long

The clone is too deep for Windows' MAX_PATH limit. Delete `.venv`, re-clone somewhere short such as `C:\dev\makeyourtree`, and install again.

ModuleNotFoundError: No module named 'makeyourtree', but pytest passes

The same MAX_PATH failure, which rolls back partially. `pytest` still passes because its configuration adds `src/` to the path, masking the missing install.

cannot be loaded because running scripts is disabled (Windows)

PowerShell execution policy. Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again.

ensurepip is not available (Linux)

Install `python3.12-venv` on Debian and Ubuntu.

The environment will not solve; pyside6 not found

You are on the `defaults` channel. Prefer `conda-forge` (Chapter 2).

Qt appears twice, or the environment is enormous

A `pip install -e .` without `-no-deps` inside a conda environment. Remove the environment and redo the install step.

B.2 Files and parsing

“My file has problems” but it looks fine

Most diagnostics are informational. The commonest, `newick.internal-label-ambiguous`, simply records that numeric internal labels were read as support values — which is nearly always right. Open `Ctrl+D` and read them; a clean file often produces one or two.

Support values appear as node names, or vice versa

The same ambiguity, resolved the other way. Numeric internal labels are read as support by default; pass `internal_labels='name'` through the API if yours are genuinely names.

Annotation rows do not match any tip

Names must match the tree exactly. The import wizard shows a match report before committing — read it. Common causes are underscores against spaces, trailing whitespace, and accession prefixes present in one file but not the other.

A tip shows a zero where it should show nothing

Check the source data: an empty field or `-` means *no value* and draws a gap. If you see a bar of zero height instead, the file really does contain a zero.

B.3 Figures

All the text is empty boxes

Qt found no fonts. This happens under the `offscreen` platform, which reports zero font families. Clear `QT_QPA_PLATFORM`, or use `xvfb-run` on a headless machine. Vector SVG from the core is unaffected because it stores text as text.

There is a gap between the tips and the annotation ring

In a circular *phylogram* this is correct: tips genuinely sit at different radii. Turn on **Align tips** () to close it.

Tracks in an unrooted layout look wrong

They are. An unrooted layout has no tip ordering for band space to use, so tracks are drawn as an unaligned column. Use a rooted layout (Section 6.6.1).

The heatmap ignores my palette setting

The heatmap has no `palette` option; its ramp is set by `color_min`, `color_mid` and `color_max`. Unknown options are recorded as diagnostics rather than applied.

Labels are clipped at the edge of the figure

Increase `width`, or limit label length with `max_label_width`.

The figure is unreadably tall

Reduce `row_spacing`, turn off tip labels, or collapse clades ()

B.4 Performance

The window freezes when I change layout on a big tree

Known limitation. Above a few thousand tips, switching layout takes several seconds during which the window does not repaint. It is working, not hung, and the operation is not currently cancellable.

Rendering a very large tree produces an enormous SVG

Expected: a 100,000-tip tree is about 28 MB of SVG. Export PNG at a chosen scale instead if the file size matters more than resolution independence.

B.5 Reporting a problem

Please open an issue with your platform, your Python version, the full error output, and — if you can share it — a small tree that reproduces the problem. A synthetic tree that shows the same failure is ideal, since it avoids sharing unpublished data.